Organización de Lenguajes y Compiladores

Ing. Mario Ramírez Tobar

01

Traducción Dirigida por la sintaxis

- Definiciones dirigidas por la sintaxis.
- Construcción de árboles.
- Evaluación ascendente de atributos sintetizados y heredados.
- Y más

02

Comprobación de Tipos

- Comprobación estática
- Comprobación dinámica
- Equivalencia de expresiones de tipos
- Conversión de tipos
- Sobrecarga de funciones y operadores

03

Generación de código Intermedio

- Lenguajes intermedios
- Árboles de sintaxis
- Código de tres direcciones

04

Assembler

- Generar código de bajo nivel
- Compilar WebAssembly

QR

Grupo de Telegram



¿QUÉ ES?

Son técnicas basadas en relacionar atributos a símbolos gramaticales, y acciones a las producciones de una gramática.
Útiles en la implementación de pequeños lenguajes para tareas especializadas.

Las usaremos para comprobar tipos, validaciones semánticas, creación de árboles de sintaxis abstracta y generación de código intermedio, entre otros.

DEFINICIONES DIRIGIDAS POR SINTAXIS

Es una gramática libre de contexto, junto con atributos y reglas.

DEFINICIONES S-ATRIBUIDAS

Las definiciones S atribuidas son definiciones que contienen solamente atributos sintetizados.

DEFINICIONES L-ATRIBUIDAS

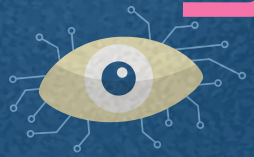
Las definiciones L atribuidas son definiciones que contienen atributos heredados y posiblemente algunos sintetizados.

Árboles de sintaxis abstracta

Es una representación gráfica de la derivación de una cadena de entrada en una gramática determinada.
Permiten que la traducción se separe del análisis sintáctico.

ATRIBUTOS SINTETIZADOS

DEFINICIONES S-ATRIBUIDAS



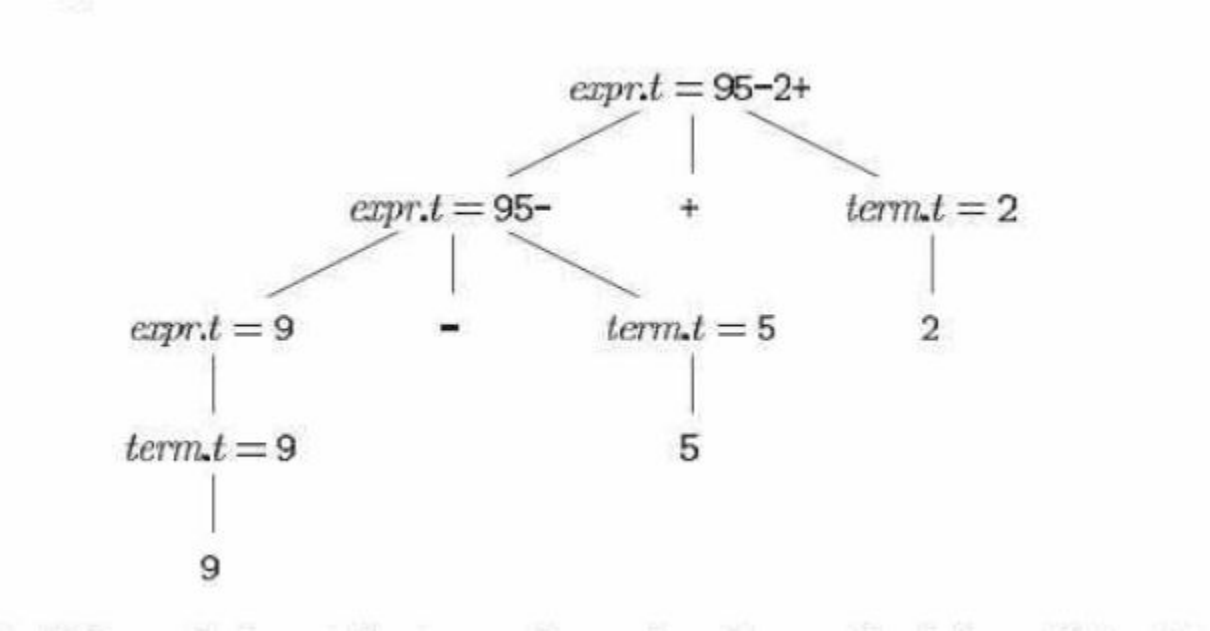


¿Que es?

Atributos que se enlazan a un símbolo gramatical generado por sus hijos al recorrer el árbol sintáctico en post orden.

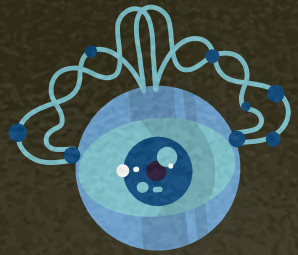
Los atributos sintetizados pueden ser aplicados en gramáticas compatibles tanto con análisis sintáctico ascendente como con descendente.

Árbol sintactico



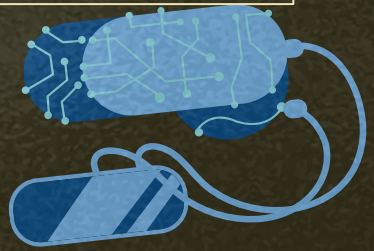
REGLAS SEMANTICAS

PRODUCCIÓN	REGLAS SEMÁNTICAS
$expr \rightarrow expr_1 + term$	$expr.t = expr_1.t \,\|\|\, term.t \,\|\|\, '+'$
$expr \rightarrow expr_1 - term$	$expr.t = expr_1.t \,\|\|\, term.t \,\|\|\, '-'$
$expr \rightarrow term$	$expr.t = term.t$
$term \rightarrow 0$	$term.t = '0'$
$term \rightarrow 1$	$term.t = '1'$
...	...
$term \rightarrow 9$	$term.t = '9'$



Ejercicio

Escribir un esquema de traducción que calcule el resultado de una expresión que acepta la operación multiplicación en enteros.



RESPUESTA 1

S -> E	Imprimir(E.val)
E -> num	E.val = num.val
E -> E1 * num	E.val = E1.val * num.val

Respuesta 2

S -> E	Imprimir(E.val)
E -> T	E.val = T.val
T -> T1 * F	T.val = T1.val * F.val
T -> F	T.val = F.val
F -> num	F.val = num.val

ATRIBUTOS HEREDADOS



DEFINICIONES L-ATRIBUIDAS



¿Que es?

Atributos que se enlazan a un símbolo gramatical y que son definidos antes de que un símbolo gramatical sea analizado, esto al recorrer el árbol sintáctico en pre-orden.

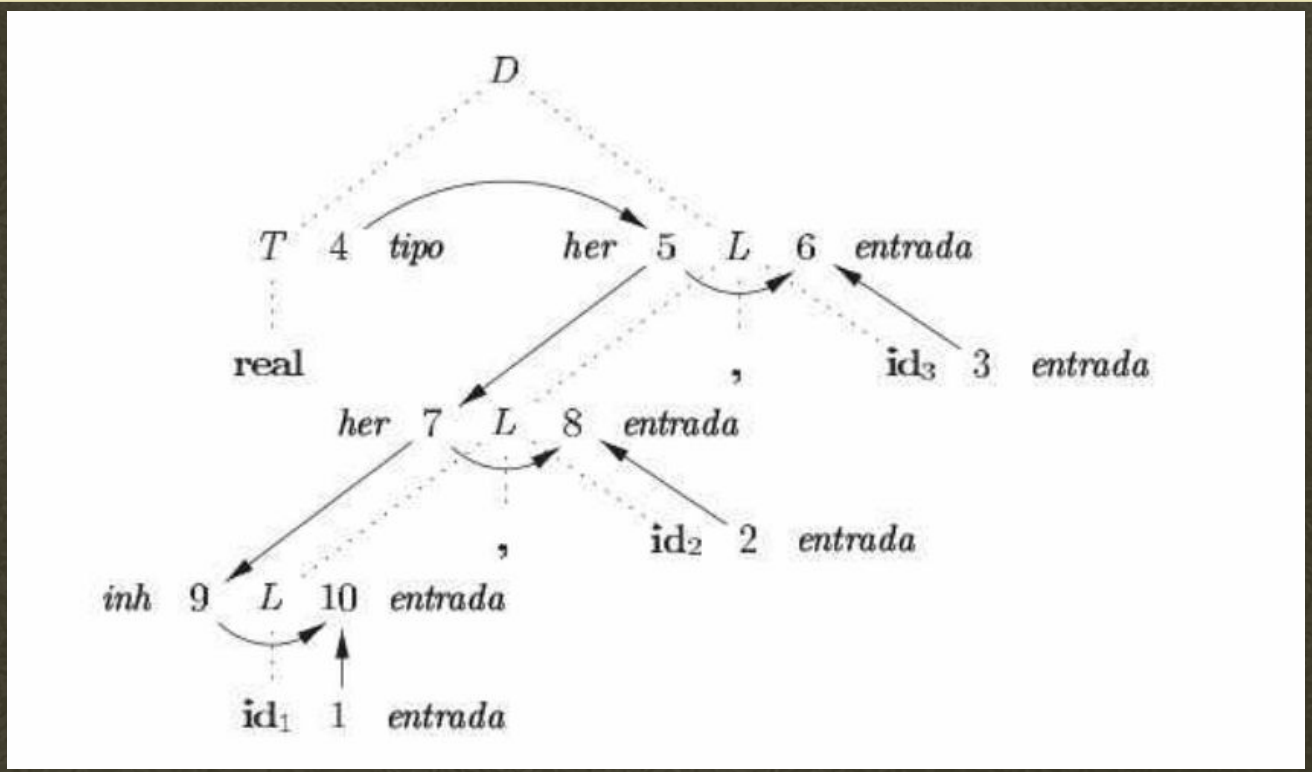
¿Que es?

Se llaman heredados porque los valores de dichos atributos son calculados antes de entrar al modo de derivación del símbolo gramatical que recibe el valor.

¿Que es?

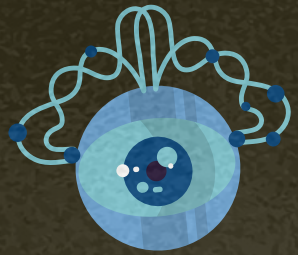
Cada acción semántica se convierte en una hoja del nodo que representa la derivación de la cadena de entrada.

Se requiere de una gramática compatible con la derivación descendente para poder aplicar atributos heredados.


D
T 4 tipo her 5 L 6 entrada
real
,
id3 3 entrada
her 7 L 8 entrada
,
id2 2 entrada
inh 9 L 10 entrada
id1 1 entrada

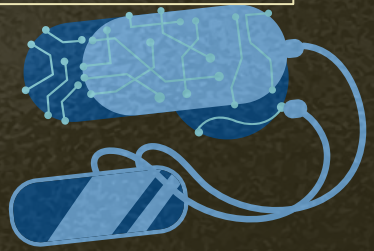
REGLAS SEMÁNTICAS

	PRODUCCIÓN	REGLAS SEMÁNTICAS
1)	$D \rightarrow T\ L$	$L.her = T.tipo$
2)	$T \rightarrow \mathbf{int}$	$T.tipo = \text{integer}$
3)	$T \rightarrow \mathbf{float}$	$T.tipo = \text{float}$
4)	$L \rightarrow L_1\ ,\ \mathbf{id}$	$L_1.her = L.her$ $agregarTipo(\mathbf{id}.entrada,\ L.her)$
5)	$L \rightarrow \mathbf{id}$	$agregarTipo(\mathbf{id}.entrada,\ L.her)$



Ejercicio

Escribir un esquema de traducción que calcule el resultado de una expresión que acepta la operación multiplicación en enteros.



RESPUESTA

S ->	T	Imprimir(T.total)
T ->	F	T'.valh = F.val
	T'	T.total = T'.total
T' ->	*	
	F	T'1.valh = T'.valh * F.val
	T'1	T'.total = T'1.total
T' ->	lambda	T'.total = T'.valh
F ->	num	F.val = num.val

RESPUESTA

S ->	T	Imprimir(T.total)
T ->	F	F.val -> T'.valh
	T'	T.total = T'.total
T' ->	*	
	F	T'.valh * F.val -> T'1.val
	T'1	T'.total = T'1.total
T' ->	lambda	T'.total = T'.valh
F ->	num	F.val = num.val

Ejercicio

Lenguaje con expresiones aritméticas con los operadores + y *





Respuesta

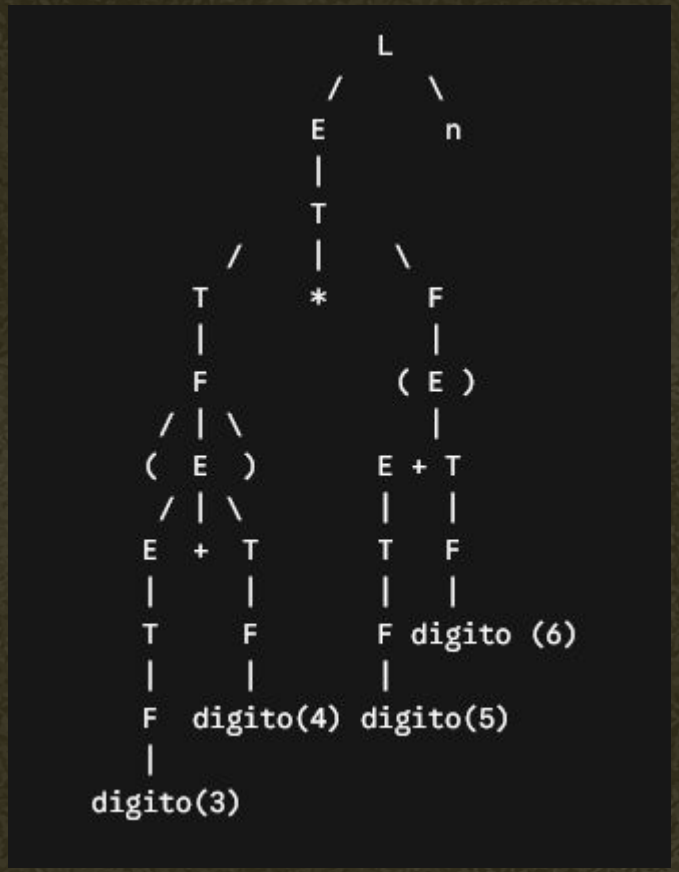
	Producción	Reglas semánticas
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digito**	$F.val =$ **digito**.lexval

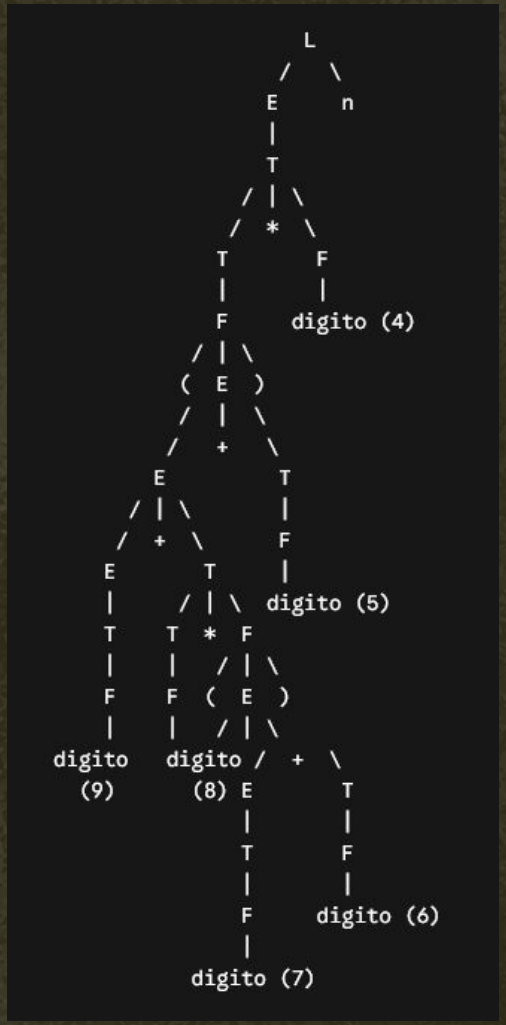
En la definición dirigida por la sintaxis anterior, proporcione árboles
de análisis sintáctico anotados para las siguientes expresiones





1) (3 + 4) * (5 + 6) n
2) (9 + 8 * (7 + 6) + 5) * 4 n


L
E
n
T
T
*
F
F
(E)
(E)
E + T
E + T
T
F
F
F
digito (6)
digito(4)
digito(5)
digito(3)


L
E
n
T
T
*
F
F
digito (4)
(E)
E
T
+
F
digito (5)
T
T
*
F
F
(E)
digito
(9)
digito
(8)
+
E
T
T
F
F
digito (6)
digito (7)

Ahora volvamos y hagamos que esta permita las expresiones del ejercicio anterior

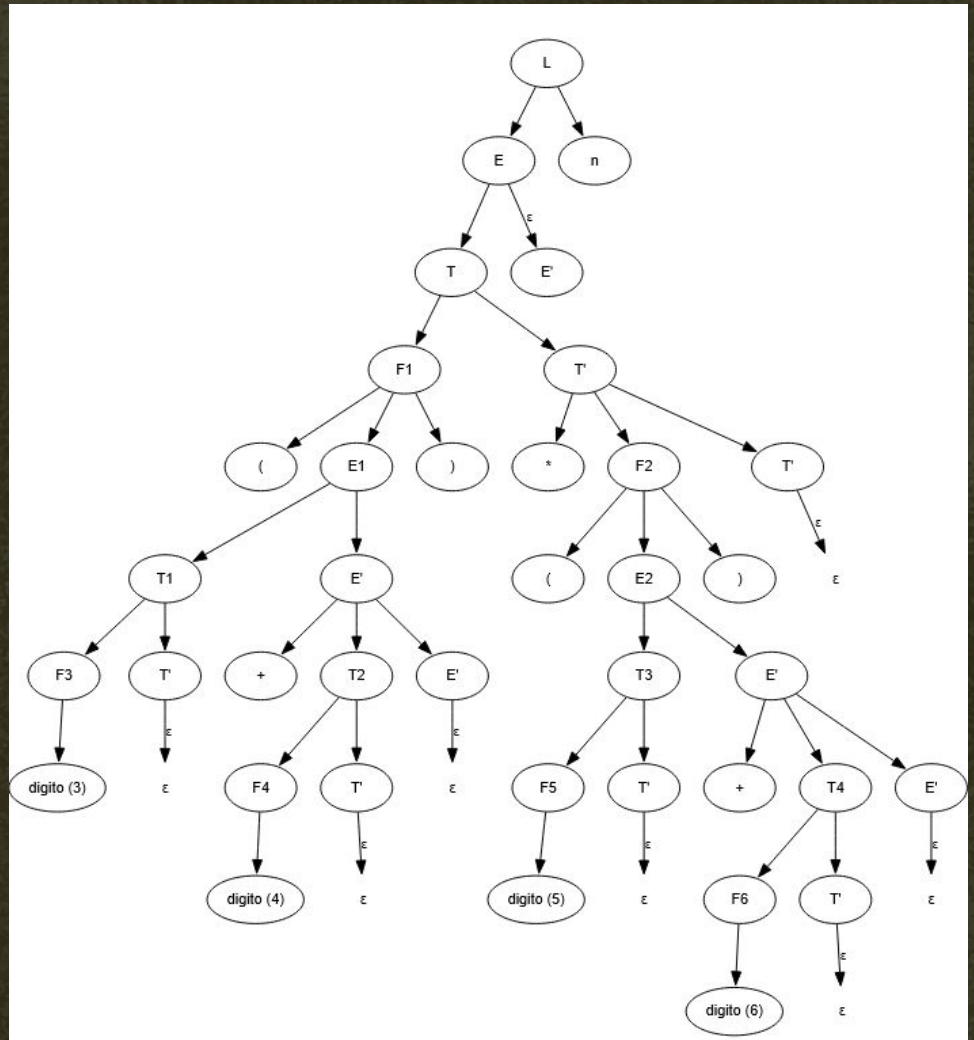




	Producción	Reglas semánticas
1)	$T \rightarrow F\ T'$	$T'.her = F.val$ $T.val = T'.sin$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.her = T'.her \times F.val$ $T'.sin = T'_1.sin$
3)	$T' \rightarrow \epsilon$	$T'.sin = T'.her$
4)	$F \rightarrow \mathbf{digito}$	$F.val = \mathbf{digito}.valex$

Respuesta

#	Producción	Reglas Semánticas
1	L -> E	L.val = E.val
2	E -> T E'	E'.her = T.val E.val = E'.sin
3	E' -> + T E'1	E'1.her = E'.her + T.val E'.sin = E'1.sin
4	E' -> epsilon	E'.sin = E'.her
5	T -> F T'	T'.her = F.val T.val = T'.sin
6	T' -> * F T'1	T'1.her = T'.her * F.val T'.sin = T'1.sin
7	T' -> \epsilon	T'.sin = T'.her
8	F -> (E)	F.val = E.val
9	F -> digito	F.val = digito.valex


L
E
n
ε
T
E'
F1
T'
(
E1
)
*
F2
T'
ε
T1
E'
(
E2
)
ε
F3
T'
+
T2
E'
T3
E'
ε
digito (3)
ε
F4
T'
ε
F5
T'
+
T4
E'
ε
digito (4)
ε
digito (5)
ε
F6
T'
ε
digito (6)
ε

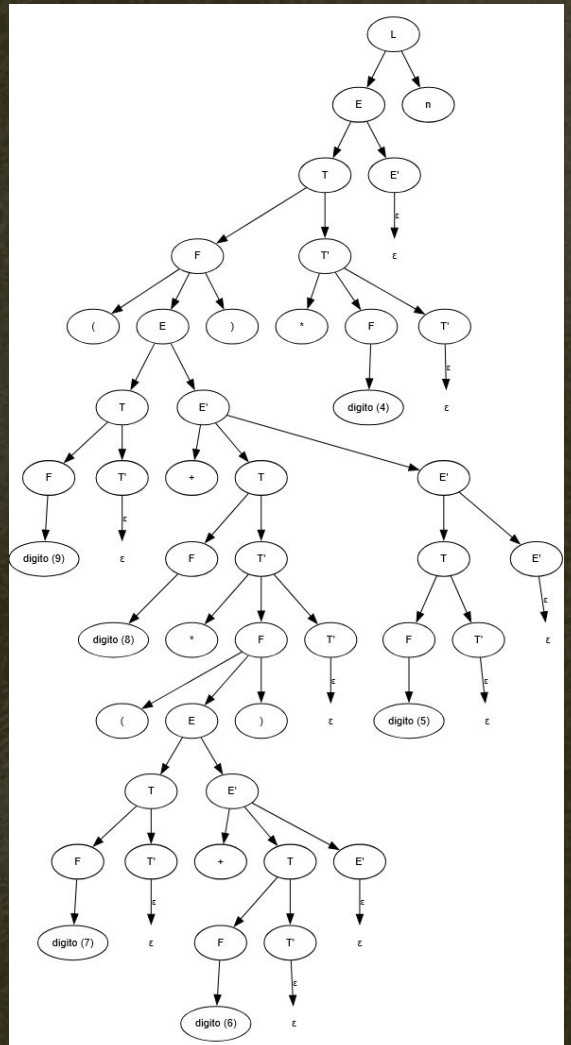
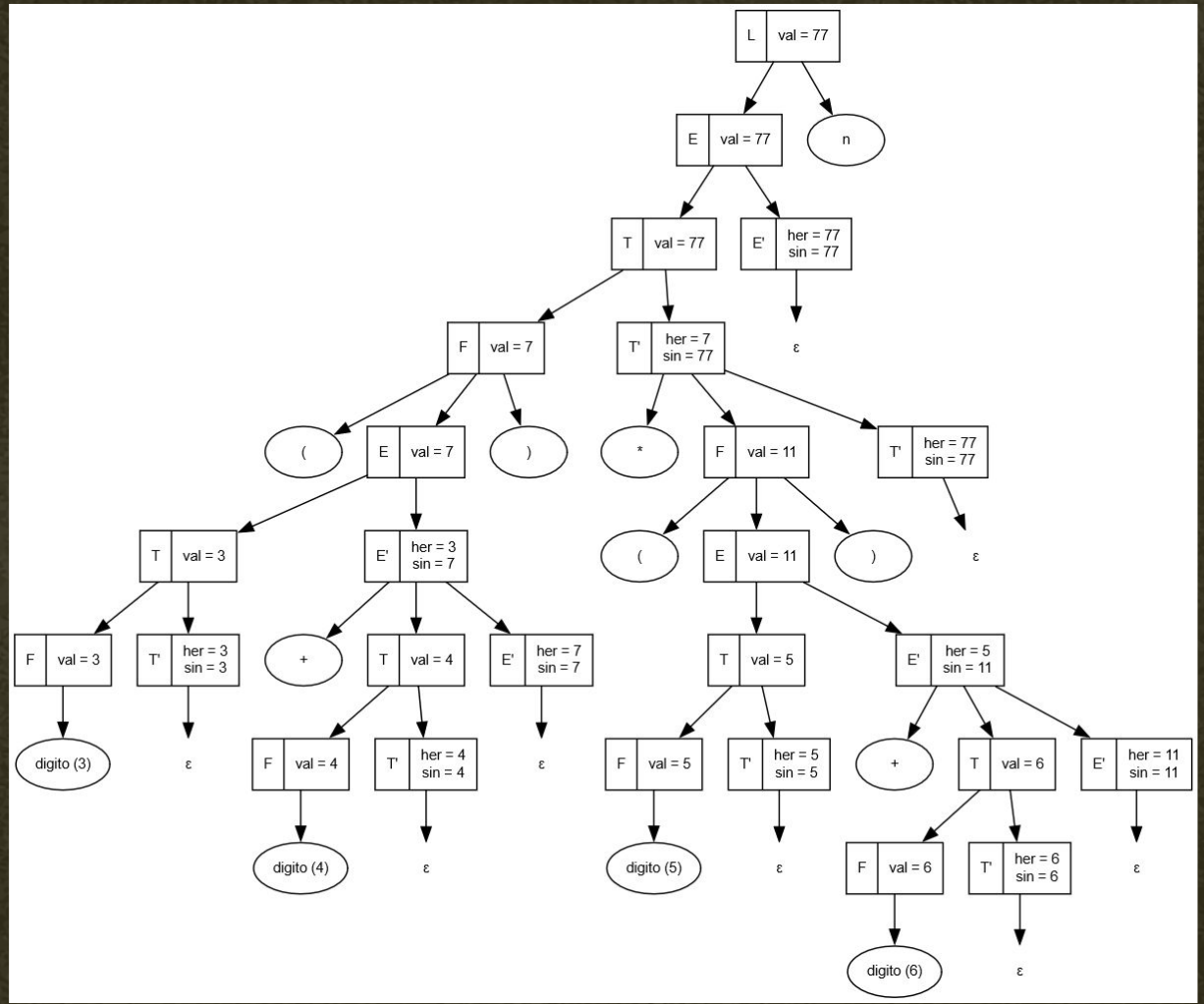

L
E
n
T
E'
ε
F
T'
ε
(
E
)
*
F
T'
ε
T
E'
digito (4)
ε
F
T'
+
T
E'
ε
digito (9)
ε
F
T'
T
E'
ε
digito (8)
*
F
T'
F
T'
ε
ε
(
E
)
ε
digito (5)
ε
T
E'
F
T'
+
T
E'
ε
digito (7)
ε
F
T'
ε
ε
digito (6)
ε

Gráfico de Dependencias






L
val = 77
E
val = 77
n
T
val = 77
E'
her = 77
sin = 77
F
val = 7
T'
her = 7
sin = 77
ε
(
E
val = 7
)
*
F
val = 11
T'
her = 77
sin = 77
T
val = 3
E'
her = 3
sin = 7
(
E
val = 11
)
ε
F
val = 3
T'
her = 3
sin = 3
+
T
val = 4
E'
her = 7
sin = 7
T
val = 5
E'
her = 5
sin = 11
digito (3)
ε
F
val = 4
T'
her = 4
sin = 4
ε
F
val = 5
T'
her = 5
sin = 5
+
T
val = 6
E'
her = 11
sin = 11
digito (4)
ε
digito (5)
ε
F
val = 6
T'
her = 6
sin = 6
ε
digito (6)
ε

Eliminar recursividad por la izquierda





¿Porque eliminarlo?

Porque ninguna gramática con recursividad por la izquierda puede analizarse sintácticamente en forma determinista

parseE(): parseE() // para reconocer la regla E -> E + T

Para reconocer E, primero necesita reconocer E... que a su vez necesita reconocer E... y así

¿Cómo la eliminamos?

En primer lugar, consideremos el caso simple, en donde lo único que nos importa es el orden en el que se realizan las acciones en un esquema de traducción orientado a la sintaxis. Por ejemplo, si cada acción sólo imprime una cadena, sólo nos preocupa el orden en el que se imprimen las cadenas. ¿Entonces qué podemos hacer?

Pasos para eliminarlo

- Al transformar la gramática, trate a las acciones como si fueran símbolos terminales.

El “truco” para eliminar la recursividad por la izquierda es tomar dos producciones:

A -> A α | β

generen cadenas que consisten en una β y cualquier número de αs y sustituirlas por producciones que generen las mismas cadenas, usando un nuevo no terminal

A -> β R
R -> α R | e

Si β no empieza con A, entonces A ya no tiene una producción recursiva por la izquierda.

Ejemplo





$$E \rightarrow E + T \{ \text{print}('+'); \}$$
$$E \rightarrow T$$

α = + T { print(‘+’): }

E ⟶ T R

R ⟶ + T { print(‘+’); } R

R ⟶ e

Ejercicio



$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow (E)$

$E \rightarrow T$

$T \rightarrow numero$



$\alpha_1 = + T \quad | \quad \alpha_2 = - T$

$\beta_1 = T \quad | \quad \beta_2 = (E)$

--- R puede ser representado por cualquier letra en producción, en este caso E' ---

E -> (E) E'

E -> T E'

E' -> + T E'

E' -> - T E'

E' -> ε

T -> número

STACK



La pila



¿Cómo funciona el stack?

Es una estructura de datos de tipo LIFO (último en entrar, primero en salir) que almacena los estados y símbolos gramaticales. Controla el progreso del análisis, guía la toma de decisiones del autómata y asegura el orden jerárquico de las estructuras del código fuente.

¿Cuáles son sus principales funciones?

- **Seguimiento del estado:** Guarda los estados actuales del autómata para saber en qué parte de la gramática se encuentra el compilador.
- **Emparejamiento de estructuras:** Ayuda a verificar que los bloques de código tengan su inicio y fin correctos, como llaves {} o paréntesis ().
- **Control de reducción:** Permite aplicar reglas de producción para transformar los símbolos de entrada en un árbol sintáctico.

¿Cómo funcionan?

- **En analizadores descendentes (LL):** Almacena los símbolos que aún faltan por procesar y predecir de la gramática.

- **En analizadores ascendentes (LR):** Guarda una combinación de estados y símbolos ya leídos mientras espera encontrar una regla para reducir.

Ejemplo

otra vez



$$E \rightarrow E + T$$
$$E \rightarrow T$$
$$T \rightarrow n$$



entrada: $n + n$

LL

$E \rightarrow T\ R$

$R \rightarrow +\ T\ R$

$R \rightarrow e$

$T \rightarrow n$

[E, $]	n + n $	inicio
[T, R, $]	n + n $	Derivar/Expandir (E⟶TR)
[n, R, $]	n + n $	Derivar/Expandir (T⟶n)
[R, $]	+ n $	Emparejar (Match)

LL

[+, T, R, $]	+ n $	Derivar/Expandir (R⟶+TX)
[T, R, $]	n $	Emparejar (Match)
[n, R, $]	n $	Derivar/Expandir (T⟶n)
[R, $]	$	Emparejar (Match)
[$]	$	Derivar/Expandir (R⟶e)
[$]	$	Aceptar

LR

[$]	n + n $	Inicio
[$, n]	+ n $	Shift (Desplazar)
[$, T]	+ n $	Reduce (T→n)
[$, E]	+ n $	Reduce (E→T)
[$, E, +]	n $	Shift (Desplazar)

LR

[$, E, +, n]	$	Shift (Desplazar)
[$, E, +, T]	$	Reduce (T⟶n)
[$, E]	$	Reduce (E⟶E+T)
[$, E]	$	Aceptar

Esquemas de Traducción



Orientado por la sintaxis



¿Que son?

Los esquemas de traducción orientados a la sintaxis postfijo pueden implementarse durante el análisis sintáctico LR mediante la ejecución de las acciones cuando ocurren las reducciones.

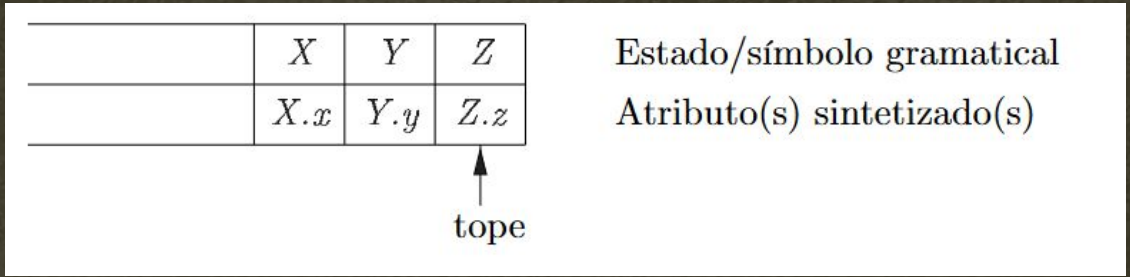
El (los) atributo(s) de cada símbolo gramatical puede(n) colocarse en la pila, en un lugar en el que puedan encontrarse durante la reducción. El mejor plan es colocar los atributos junto con los símbolos gramaticales (o con los estados LR que representan estos símbolos) en registros en la misma pila.

Sintetizados y Postfijo

Implementación de esquemas postfijo

Los esquemas de traducción orientados a la sintaxis postfijo pueden implementarse durante el análisis sintáctico LR mediante la ejecución de las acciones cuando ocurren las reducciones.

La pila del analizador sintáctico contiene registros con un campo para un símbolo gramatical (o estado del analizador sintáctico) y, debajo de él, un campo para un atributo. Los tres símbolos gramaticales X Y Z están en la parte superior de la pila; tal vez estén a punto de reducirse en base a una producción como A → X Y Z.



Todos sintetizados

Si los atributos sintetizados son todos sintetizados, y las acciones ocurren en los extremos de las producciones, entonces podemos calcular los atributos para el encabezado cuando reduzcamos el cuerpo al encabezado. Si reducimos mediante una producción como $A \rightarrow X\ Y\ Z$, entonces tenemos todos los atributos de X, Y y Z disponibles, en posiciones conocidas de la pila.

Con Acciones dentro de producciones

Puede colocarse una acción en cualquier posición dentro del cuerpo de una producción. Por ende, si tenemos una producción B → X {a } Y, la acción a se realiza después de que reconozcamos a X.

- Si el análisis sintáctico es ascendente, entonces ejecutamos la acción a tan pronto como aparezca esta ocurrencia de X en el tope de la pila de análisis sintáctico.
- Si el análisis sintáctico es descendente, ejecutamos la acción a justo antes de tratar de expandir esta ocurrencia de Y (si Y es una no terminal) o comprobamos a Y en la entrada (si Y es una terminal).

¿Cómo se puede implementar cualquier esquema?

Cualquier esquema de traducción orientado a la sintaxis puede implementarse de la siguiente manera:

1. Ignorando las acciones, se analiza sintácticamente la entrada y se produce un árbol de análisis sintáctico como resultado.
2. Después se examina cada nodo interior N, por ejemplo, uno para la producción $A \rightarrow \alpha$. Se agregan hijos adicionales a N para las acciones en α, para que los hijos de N de izquierda a derecha tengan exactamente los símbolos y las acciones de α.
3. Realizar un recorrido en preorden (vea la sección 2.3.4) del árbol, y tan pronto como se visite un nodo etiquetado por una acción, realizar esa acción.

Heredados y preorden.

¿Cómo se define?

Ahora, vamos a considerar el caso más general de un definición dirigida por la sintaxis con atributos heredados por la izquierda.
Si suponemos que se puede analizar de arriba hacia abajo, puede implementarse adjuntando las acciones a un árbol de análisis sintáctico y ejecutándose durante el recorrido preorden del árbol.

Las Reglas

Para convertir una definición dirigida por la sintaxis con atributos heredados por la izquierda en un esquema de traducción orientado a la sintaxis:

1. Incrustar la acción que calcule los atributos heredados para un no terminal A, justo después de esa ocurrencia de A en el cuerpo de la producción. Si varios atributos heredados para A dependen uno del otro en forma acíclica, ordenar la evaluación de los atributos, de manera que los que se necesiten primero sean los que se calculen primero.
2. Colocar las acciones que calculan un atributo sintetizado para el encabezado de una producción, al final del cuerpo de esa producción.

Ejemplo





S → while (C) S1

¿Qué hacer?

En este ejemplo de flujo de control, lo único que vamos a generar son etiquetas. Se asume que todas las demás instrucciones de código intermedio se van a generar mediante partes del esquema de traducción orientado a la sintaxis que no se muestran.

En forma específica, generaremos instrucciones explícitas de la forma etiqueta L, en donde L es un identificador, para indicar que L es la etiqueta de la instrucción que le sigue.

$$
\begin{array}{ll}
S \rightarrow \textbf{while}\ (C)\ S_1 & L1 = new(); \\
& L2 = new() \\
& S_1.siguiente = L1; \\
& C.false = S.siguiente; \\
& C.true = L2; \\
& S.codigo = \textbf{etiqueta} \parallel L1 \parallel C.codigo \parallel \textbf{etiqueta} \parallel L2 \parallel S_1.codigo
\end{array}
$$

¿Que hace?

Esta definición dirigida por la sintaxis tiene atributos heredados por la izquierda. Al convertirla en un esquema de traducción orientado a la sintaxis, lo único que queda pendiente es cómo manejar las etiquetas L1 y L2, que son variables y no atributos. Si tratamos a las acciones como no terminales falsos, entonces dichas variables pueden tratarse como los atributos sintetizados de los no terminales falsos. Como L1 y L2 no dependen de ningún otro atributo, pueden asignarse a la primera acción en la producción.

$$
\begin{array}{lll}
S \rightarrow \textbf{while} \; (& \{ \; L1 = new(); \; L2 = new(); \; C.false = S.siguiente; \; C.true = L2; \; \} \\
\quad C \;) & \{ \; S_1.siguiente = L1; \; \} \\
\quad S_1 & \{ \; S.codigo = \textbf{etiqueta} \parallel L1 \parallel C.codigo \parallel \textbf{etiqueta} \parallel L2 \parallel S_1.codigo; \; \}
\end{array}
$$

Implementación de Traducción

Orientado por la sintaxis





Descenso recursivo

Cómo se trabaja

Un analizador sintáctico de descenso recursivo tiene una función A para cada no terminal A.

A. Los argumentos de la función A son los atributos heredados del no terminal A.
B. El valor de retorno de la función A es la colección de atributos sintetizados del no terminal A.

¿Qué hacer con la función?

1. Decidir la producción que utilizaremos para expandir A.
2. Comprobar que cada terminal aparezca en la entrada cuando se le requiera.
3. Preservar en las variables locales los valores de todos los atributos necesarios para calcular los atributos heredados para los no terminales en el cuerpo, o los atributos sintetizados para el no terminal del encabezado.
4. Llamar a las funciones correspondientes a los no terminales en el cuerpo de la producción seleccionada, proporcionándoles los argumentos apropiados. Como la definición dirigida por la sintaxis subyacente tiene atributos heredados por la izquierda, ya hemos calculado estos atributos y los almacenamos en variables locales.

Ejemplo

```
cadena S(etiqueta siguiente) {
    cadena Scodigo, Ccodigo; /* las variables locales contienen fragmentos de código */
    etiqueta L1, L2; /* las etiquetas locales */
    if ( entrada actual == token while ) {
        avanzar entrada;
        comprobar que siga '(' en la entrada, y avanzar;
        L1 = new();
        L2 = new();
        Ccodigo = C(siguiente, L2);
        comprobar que siga '(' en la entrada, y avanzar;
        Scodigo = S(L1);
        return("etiqueta" || L1 || Ccodigo || "etiqueta" || L2 || Scodigo);
    }
    else /* otros tipos de instrucciones */
}
```

Llevar todo el texto puede ser ineficiente

```
void S(etiqueta siguiente) {
        etiqueta L1, L2;  /* las etiquetas locales */
        if ( entrada actual == token while ) {
                avanzar entrada;
                comprobar que siga '(' en la entrada, y avanzar;
                L1 = new();
                L2 = new();
                print("etiqueta", L1);
                C(siguiente, L2);
                comprobar que siga ')' en la entrada, y avanzar;
                print("etiqueta", L2);
                S(L1);
        }
        else /* otros tipos de instrucciones */
}
```

Ascendente

¿Qué podemos hacer?

Podemos adaptar la gramática para calcular la misma definición dirigida por la sintaxis en la nueva gramática, durante un análisis sintáctico LR. El “truco” tiene tres partes:

1. Empezar con el esquema de traducción orientado a la sintaxis el cual coloca acciones incrustadas antes de cada no terminal.
2. Introducir en la gramática un no terminal como marcador en vez de cada acción incrustada. Cada una de estas posiciones obtiene un marcador distinto, y hay una producción para cualquier marcador M, en específico, $M \rightarrow e$.
3. Modificar la acción a si el no terminal como marcador M la sustituye en alguna producción $A \rightarrow \alpha \{a\} \beta$, y asociar con $M \rightarrow$ una acción a ′ que:
    a. Copie, como atributo heredado de M, cualquier atributo de A o símbolo de α que la acción a requiera.
    b. Calcule los atributos de la misma forma que a, pero que los haga atributos sintetizados de M.

Ejemplo

$$
\begin{aligned}
S &\rightarrow \textbf{while}\ (\ M\ C\ )\ N\ S_1 \\
M &\rightarrow \epsilon \\
N &\rightarrow \epsilon
\end{aligned}
$$

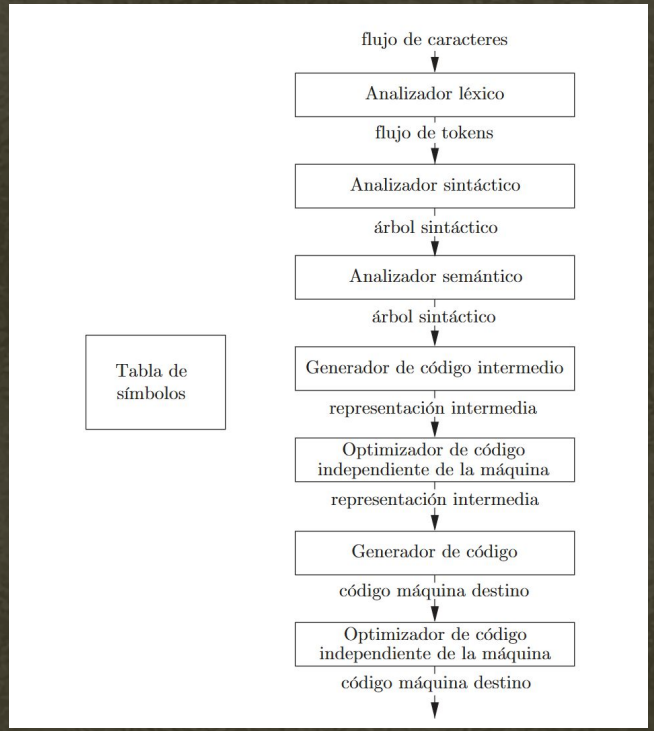
Análisis Semántico



¿Tiene mi lenguaje sentido?



Estructura de un Compilador



¿Por qué?

Se evalúan si las sentencias escritas en algún lenguaje tienen significado válido dentro de las reglas de dicho lenguaje.

Recordemos el Analizador Léxico

A la primera fase de un compilador se le llama análisis de léxico o escaneo. El analizador de léxico lee el flujo de caracteres que componen el programa fuente y los agrupa en secuencias significativas, conocidas como lexemas. Para cada lexema, el analizador léxico produce como salida un token de la forma:

⟨nombre-token, valor-atributo⟩

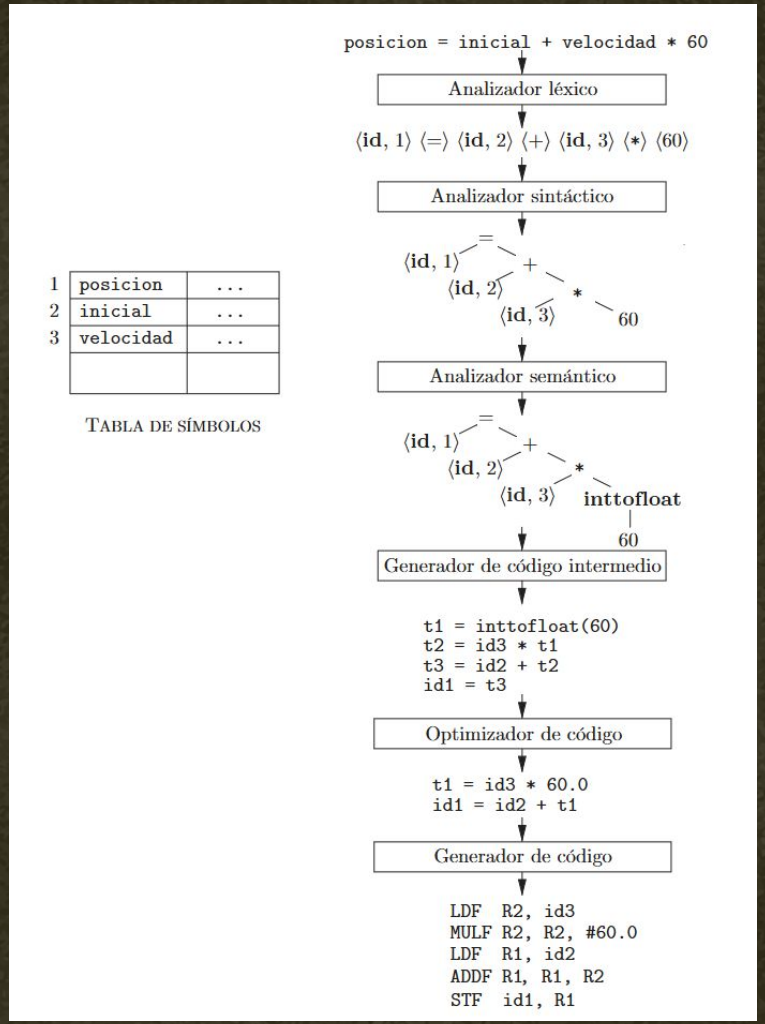
que pasa a la fase siguiente, el análisis de la sintaxis. En el token, el primer componente nombre-token es un símbolo abstracto que se utiliza durante el análisis sintáctico, y el segundo componente valor-atributo apunta a una entrada en la tabla de símbolos para este token. La información de la entrada en la tabla de símbolos se necesita para el análisis semántico y la generación de código.

Ejemplo





posicion = inicial + velocidad * 60


posicion = inicial + velocidad * 60
Analizador léxico
⟨id, 1⟩ ⟨=⟩ ⟨id, 2⟩ ⟨+⟩ ⟨id, 3⟩ ⟨*⟩ ⟨60⟩
Analizador sintáctico
Analizador semántico
inttofloat
Generador de código intermedio
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
Optimizador de código
t1 = id3 * 60.0
id1 = id2 + t1
Generador de código
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
1 posicion ...
2 inicial ...
3 velocidad ...
TABLA DE SÍMBOLOS

¿Entonces qué es el análisis Semántico?

El analizador semántico utiliza el árbol sintáctico y la información en la tabla de símbolos para comprobar la consistencia semántica del programa fuente con la definición del lenguaje. También recopila información sobre el tipo y la guarda, ya sea en el árbol sintáctico o en la tabla de símbolos, para usarla más tarde durante la generación de código intermedio.

Comprobaciones Semánticas

En un lenguaje fuertemente tipado lo siguiente no tiene significado válido.

```
int x = "aaa";
```

Comprobaciones Semánticas

En un lenguaje fuertemente tipado lo siguiente no tiene significado válido.

```
public void algo() {
}
int x = algo();
```

Comprobaciones Semánticas

Existen comprobaciones que se hacen en tiempo de compilación, o comprobaciones estáticas.

Existen comprobaciones que se hacen en tiempo de ejecución, o comprobaciones dinámicas.

Nos centraremos en comprobaciones estáticas.

¿Cuáles son?

- Comprobación de tipos
- Comprobación de nombres
- Comprobación de ámbitos
- Comprobación de flujo de control
- Comprobación de unicidad
- Comprobación de accesos

Comprobación de Tipos

Por ejemplo:

int x = “hola” + 25;

No es válido ya que el resultado de la suma (concatenación) es una cadena.

Comprobación de nombres

Por ejemplo:

```
int x = 15;
int y = z + x;
```

No es válido ya que la variable z no existe

Comprobación de ámbitos

```
public void sumar(int a, int b) {
    if (a > b ) {
        int x;
    }
    x = 15;
}
```

No es válido ya que la variable x fue declarada en un ámbito diferente

Comprobación de flujo de control

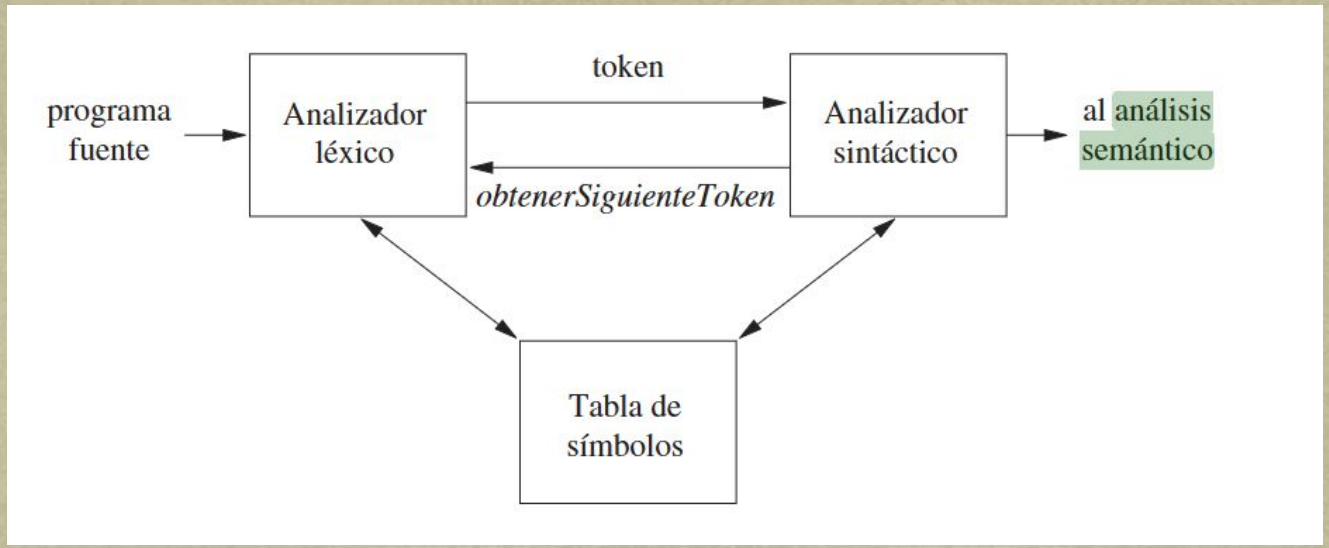
```
public void sumar(int a, int b) {
    if (a > b ) {
        break;
    }
}
```

No es válido ya que la palabra break no se aplicable en la instrucción if, solo en ciclos o en switches.

Comprobación de acceso

```
public class Clase1 {
	private int x;
}
public class Clase2 {
	public void algo() {
		Clase1 c1 = new Clase1();
		C1.x = 25;
	}
}
```

No es válido ya que el miembro x de Clase1 es privado


programa
fuente
Analizador
léxico
token
obtenerSiguienteToken
Analizador
sintáctico
al análisis
semántico
Tabla de
símbolos

Comprobación de Tipos

¿Que es?

La comprobación de tipos estática se da en tiempo de compilación, y es la encargada de verificar que los diferentes valores expresados en un lenguaje sean operados solo con otros valores del mismo tipo.

Ejemplos

Por ejemplo:

int x = “algo”;

No es válido ya que la literal “algo” es de tipo string y la variable es de tipo entera.

Ejemplos

```
int x = 25.25;

int x = 25 + 0.12;

int x = “hola” + 25;
```

¿Porque esto es válido?

String x = “hola” + 25;

1. El operador + está sobrecargado.
2. Al concatenar, se realiza una “conversión” del literal 25 a cadena.

Conversión implícita de tipos

Los lenguajes con tipado estático permiten la operación entre ciertos tipos de datos sin que se de un error de tipos.
Los datos numéricos son los que tienen esta propiedad.

Tipos enteros con conversión

- long
- integer
- byte
- short
- Char
- boolean

Tipos decimales con conversión

- Float
- Double

La conversión siempre se da hacia el tipo con mayor jerarquía.

Comprobación nominal o de nombre

Se basa en el nombre y el significado asociado a ese nombre de tipo.

En Java el tipo primitivo int tiene ciertas características dentro del lenguaje que permite realizar comprobaciones y una conversión implícita cuando se opera con un float.

Comprobación estructural

Se basa en comprobar los componentes de los tipos.

Si tenemos definidos los structs:

Struct *coordenadas* { int x; int y}

Struct *otroTipo* { int x; int y}

¿Esto es válido?

```
coordenadas miVar = { 5, 6};
otroTipo   otraVar = {7, 8};

miVar = otraVar;
```

Tabla de Tipos

¿Que es?

Estructura tabular que contiene la información relevante sobre los tipos de datos del lenguaje.

Contiene los tipos propios del lenguaje pero puede ser alimentada con nuevos tipos creados por el usuario.

¿Que contiene?

- **Nombre**: identificador del tipo
- **TipoBase**: el tipo del que se deriva
- **Padre**: el tipo padre
- **Dimensión**: número de elementos
- **Mínimo**: indice mínimo
- **Maximo**: indice máximo
- **Ámbito**: contador de ámbito

¿Que contiene?

Las columnas anteriores son columnas propuestas que una tabla de tipos puede tener, sin embargo, los diseñadores del lenguaje son los que finalmente definen las columnas a utilizar dependiendo de las necesidades del lenguaje.

Tabla de Símbolos

¿Que es?

Estructura tabular que contiene la información relevante sobre los símbolos (variables, funciones, procedimientos, métodos, clases) declarados por el usuario.

¿Para qué es?

Esta tabla es usada no solo para la gestión de tipos, sino también ayuda a generar código 3 direcciones y al generar código final ya que puede almacenar la información sobre la posición en memoria de cada símbolo.

¿Que contiene?

- **Nombre**: El nombre del símbolo
- **Categoría**: Variable/método/clase, etc
- **Tipo**: El tipo del símbolo
- **Numero parámetros**: Cantidad de param
- **Lista de parámetros**: Listado de tipos de parámetros
- **Direccion**: direccion en memoria relativa
- **Ambito**: Número de ámbito si se maneja solo una tabla de símbolos.

Ejemplo





```
program p ;
        type vector = array [0 .. 7 ] of integer ;
        var x : integer ; v : vector ;

        function factorial( x : integer ) : integer;
                var y , z : integer ;
        begin
                y := 1 ;
                for z := 1 to x do y := y * z ;
                factorial := y
        end ;

        function suma( v : vector ; k : integer ) : vector;
                type vect = array [ 1 .. 8 ] of integer ;
                var x : integer ; v1 : vect ;
        begin
                        for x := 0 to k do v1[ x + 1 ] := v[ x ] ;
                        suma := v1
        end ;

begin
        x := 5 ;
        x := factorial ( x ) ;
        for x := 0 to 7 do v[ x ] := x ;
        v := suma ( v , x )
end
```

Tabla de tipos

ID	Nombre	Tipo Base	Padre	Dimension	Minimo	Maximo	Ambito
0	integer						0
1	boolean						0
2	vector	0		1	0	7	0
3	vect	0		1	1	8	suma

Tabla de símbolos

ID	Nombre	Categoría	Tipo	NumParam	ListParam	Dir	Ámbito
0	x	var	0			0	0
1	v	var	2			1	0
2	factorial	func	0	1	[0]		0
3	x	param	0			0	factorial
4	y	var	0			1	factorial
5	z	var	0			2	factorial
6	return					3	factorial
7	suma	func	2	2	[2,0]		0
8	v	param	2			0	suma

Tabla de símbolos, cont.

ID	Nombre	Categoría	Tipo	NumParam	ListParam	Dir	Ámbito
9	k	param	0			8	suma
10	x	var	0			9	suma
11	v1	var	3			10	suma
12	return					18	suma

Comprobación de Nombres

¿De que se encarga?

La comprobación de nombres estática se da en tiempo de compilación, y es la encargada de verificar que los diferentes símbolos (o nombres) de los diferentes elementos (variables, clases, métodos, etc) hayan sido definidos.

Ejemplo

```
int x = 15;
int y = z + x;
```

No es válido ya que la variable z no existe

Comprobación de Ámbitos

¿Que es?

La comprobación de ámbitos es una comprobación que está muy relacionada con la comprobación de nombres debido a que los símbolos definidos tienen cierto alcance dentro del programa.

Recordemos

El ámbito de un símbolo define el ‘tiempo de vida’ de dicho símbolo.

Si un símbolo es definido en un bloque de código, entonces ese símbolo solo ‘vive’ dentro de ese bloque.

Ejemplo

```
public void sumar(int a, int b) {
    if (a > b ) {
        int x;
    }
    x = 15;
}
```

No es válido ya que la variable x fue declarada en un ámbito diferente

Tabla de símbolos

De nuevo es la respuesta de la estructura que nos permite hacer las comprobaciones tanto de nombres como de ámbitos.

Nos podemos apoyar también con estructuras tipo pilas para la validación de ámbitos.

Comprobación de Control de Flujo

¿Que es?

Algunas estructuras de control de flujo tienen limitantes y reglas específicas del lenguaje.

Dichas reglas se pueden validar tanto a través de reglas semánticas en la gramática o a través de árboles de sintaxis abstracta

Ejemplo

```
public void sumar(int a, int b) {
    if (a > b ) {
        break;
    }
}
```

No es válido ya que la palabra break no es aplicable en la instrucción if, solo en ciclos o en switches.

¿Cómo solucionarlo?

En el ejemplo anterior, debido a que debemos saber cual es la estructura de control en la que nos encontramos, se debe recurrir a herramientas como un árbol de sintaxis abstracta

Comprobación de Unicidad

¿Que es?

Algunas estructuras de flujo de control tienen limitantes y reglas específicas del lenguaje. Dichas reglas se pueden validar tanto a través de reglas semánticas en la gramática o a través de árboles de sintaxis abstracta

Ejemplo

```
public int sumar(int a) {
	if (a > 5) {
		return 5;
	} else {
		return 2;
	}
	return 10;
}
```

No es válido ya que para cada camino del if ya se tiene su respectivo return, y return 10 esta de mas.

Ejemplo

```
public void sumar(int a) {
	switch(a) {
		case 1:
			break;
		case 2: {
			System.out.println(“algo”);
			break;
			break;
		}
	}
}
```

No es válido ya que hay dos ocurrencias de la palabra break en el mismo bloque.

¿Cómo lo solucionamos?

En el ejemplo anterior, debido a que debemos saber cual es la estructura de control en la que nos encontramos, se debe recurrir a herramientas como un árbol de sintaxis abstracta

Comprobación de Acceso

¿Que es?

En Lenguajes de programación orientada a objetos, algunos de los miembros de clase (atributos y metodos) pueden estar sujetos a límites de acceso (public, private, etc.)

Ejemplo

```
public class Clase1 {
	private int x;
}
public class Clase2 {
	public void algo() {
		Clase1 c1 = new Clase1();
		C1.x = 25;
	}
}
```

No es válido ya que el miembro x de Clase1 es privado

¿Qué hacer?

En el ejemplo anterior, debido a que debemos saber cual es el ‘alcance’ de los miembros de una clase, podemos utilizar la tabla de símbolos para registrar dicha configuración de accesos.
Esta validación es muy similar a la comprobación de ámbitos de variables.

Análisis Semántico



¿Tiene mi lenguaje sentido?

